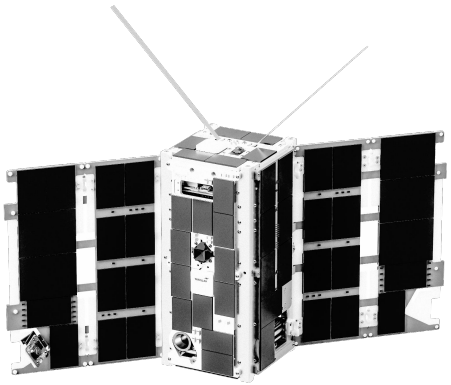
Summer 2027 - Lundeen Cahilly

Engineering Portfolio



Stanford Student Space Initiative

SAMWISE Flight GNC

SAMWISE (see above) is a 2U CubeSat designed and built by the Stanford Student Space Initiative. The mission purpose was to demonstrate autonomous attitude determination and control, deployable solar arrays, and on-orbit software updates. Developing the spacecraft hardware and flight software in house kept its total build cost per unit below $15k.

My Role

Satellites Team Co-Lead; GNC & Software Engineer. My job was to take extreme ownership of the project and get things done. When I began the ADCS work, the subsystem consisted of a populated sensor PCB without an integrated flight stack to turn its measurements into attitude estimates and actuator commands.

During summer 2025, I spent >500 hours building its flight GNC stack from the ground up while co-leading the 40-person satellite team. I implemented ~90% of the ADCS flight software and owned its architecture, integration, debugging, and validation. The resulting system could autonomously detumble, estimate attitude, command its magnetorquers and reaction wheels, and manage safety-critical mode transitions.

Foundational Driver and Hardware Development

I started off by writing 10 drivers for sensors, actuators, and board communication to allow our chip to interface with the hardware. I validated via hardware-in-the-loop testing and subsequently identified a flight-critical bug with our watchdog. I then learned to redesign and ultimately fix all boards with that circuit.

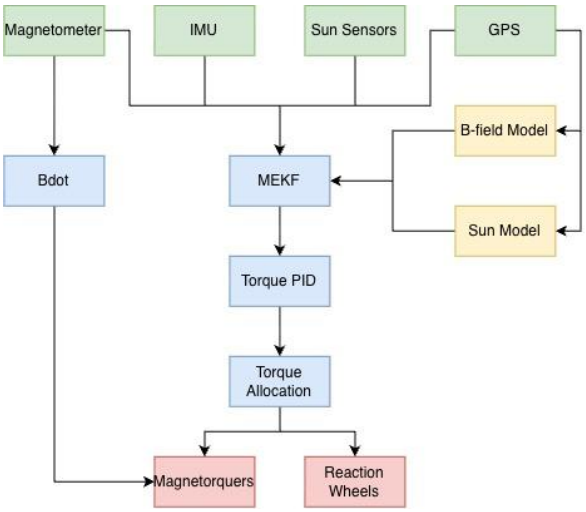


Figure 1: High-level SAMWISE GNC Flight Architecture

Attitude Determination

1. ***Multiplicative Extended Kalman Filter (MEKF):*** A conventional additive EKF updates the quaternion directly, which can violate its unit-length constraint. I instead chose to write a MEKF, which estimates a three-dimensional attitude correction while propagating with the IMU and correcting against measured and modeled Sun and B-field vectors, avoiding the footgun.
2. ***Magnetic Field Model:*** I implemented IGRF-14 from scratch in flight C++, evaluating the spherical-harmonic model using GPS position and time. The model was validated against an external implementation to sub-arcminute precision globally.
3. ***Robust sun-vector estimation:*** Our sun sensors self-occlude with the bus in many orientations, so I wrote an algorithm using RANSAC to identify and reject outliers. Validated to subdegree accuracy via a ray-tracing simulation.

Attitude Control

The mission's primary control objective was to execute B-dot detumbling, a mode we had not previously flown because of power constraints and limited validation. I validated the algorithm via both simulation and by physically spinning a bare GNC testing rig (see right) in a Helmholtz cage, after a polarity testing procedure had been performed.

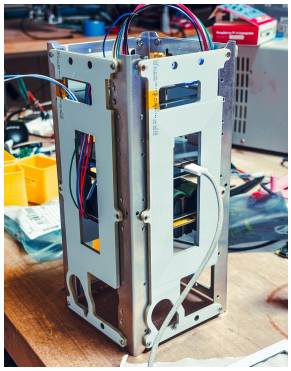


Figure 2: B-dot Testing Rig

I also developed 3 independent SISO PID controllers for reaction-wheel attitude control, one per body axis (ignores gyro), but our team decided to descope pointing when hitting bandwidth limits for supporting flight validation.

Mission success for the reaction-wheel system was thus limited to demonstrating and verifying wheel actuation.

Starguard

TreeHacks 2026 · NVIDIA Track Finalist

Starguard is a prototype for autonomous collision avoidance when a spacecraft cannot rely on GPS or ground contact. I worked with my teammates Sid Anantha, Rahul Ayanampudi, and Raj Thapliyal to repurpose star-tracker imagery to detect nearby spacecraft, recover their relative motion across frames, and plan avoidance maneuvers.

My focus for this project was building the simulation and synthetic data pipeline. I used TensorGator's CUDA backend to propagate more than 10,000 satellites through a full day in less than 10 seconds, reducing runtime from hours on a CPU to seconds on a GPU. This would allow a satellite to do massively parallel propagation, as needed. I then wrote a pinhole-camera renderer that converted trajectories into labeled star-tracker images for our downstream CV pipeline.



Figure 3: Simulated Star Tracker Images

Test-Time Failure Prediction for Spacecraft Pose Estimation

CS229 Machine Learning

Modern spacecraft pose estimators train largely on synthetic imagery, but their accuracy degrades on real images. Anjali Sreenivas and I asked whether an estimator could recognize its own failures at test time, before an unreliable pose measurement reached a navigation filter.

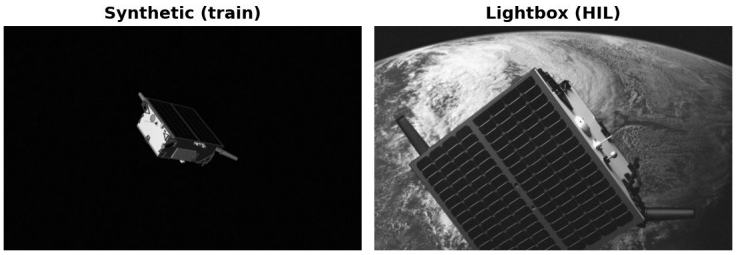


Figure 4: Synthetic vs. Hardware-in-the-Loop Images

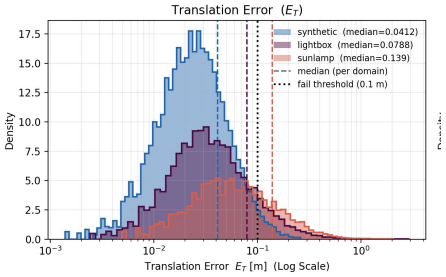


Figure 5: SPNv2 Error Across Synthetic and HIL Domains

I modified the inference pipeline of a spacecraft pose estimation model, SPNv2, to extract 13 internal confidence features from each forward pass. These included disagreement between its two pose heads, keypoint heatmap confidence, and detection confidence.

The SPEED+ dataset provides synthetic imagery and two hardware-in-the-loop domains with different lighting conditions. Using synthetic labels alone, Anjali and I trained class-weighted logistic regression classifiers to predict whether each translation or rotation estimate would exceed a defined error threshold. Applied unchanged to the real-image domains, the translation classifiers reached AUCs of 0.84 and 0.80, nearly matching classifiers trained directly on real-domain labels. This showed that the model's internal confidence signals could identify translation failures across the synthetic-to-real gap without any labeled real images. Rotation prediction was weaker because the target's symmetry often led both pose heads to agree on an incorrect orientation.

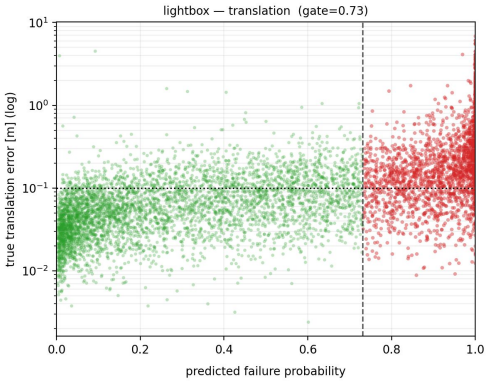


Figure 6: Our Predictor on HIL Images

In practical applications, this failure prediction model could be placed in front of a navigation filter to downweight or reject measurements that it flags as unreliable. Because the classifier transfers across the synthetic to HIL domains, it could provide that safeguard without requiring labeled imagery from the deployment environment, reducing the risk that a bad estimate corrupts the state.

K2 Flight GNC and Simulation

GNC Intern · Summer 2026

K2 Space is building 20kW and 100kW class satellites, with an emphasis on low cost and orbit flexibility. I worked on simulation and flight GNC development for the upcoming Trinity mission (Q1 2027), along with some work on Gravitas (in-orbit).



I owned end-to-end simulation and GNC development for K2's first in-house high-orbit momentum-dumping system, including dynamics modeling, control architecture trades, flight control and estimation algorithms, and validation. I also worked on a new guidance mode, implemented navigation robustness fixes, expanded MC capabilities, and more.